

INSuRE Spring 2024
Midterm Report - March 21, 2024

Assessing the Potential Impact of Memory-Safe Languages for System Software Security

Investigators:

**The University of Arizona and The University of Texas at Dallas
Team 3**

Chia-Hao Yang - Student Researcher - cxy200019@utdallas.edu

Josh McCoy - Student Researcher - mccoyja3@arizona.edu

Nate Schultz - Student Researcher - n8r@arizona.edu

Ryan Brockinton - Student Researcher - rtb180002@utdallas.edu

Bill Hayes - Faculty Support - billhayes@arizona.edu

Brian Ricks - Faculty Support - bwr031000@utdallas.edu

Problem Mentor:

Neal Ziring - National Security Agency - nlzirin@uwe.nsa.gov

Table of Contents

Table of Contents	1
1.0 Project Introduction	2
1.1 Top-Level Research Project Description	3
1.2 Project Objectives / Goals	4
1.3 Investigator Biographies	6
2.0 Research Plan and Progress to Date	7
2.1 Research Plan Summary	9
2.2 Deliverables	11
2.3 Progress to Date	13
3.0 Summary of Results/Conclusions to Date	18
4.0 Issues and Challenges	19
5.0 Bibliography	20
Appendix A: Acronyms and Terms/ Definitions	20

1.0 Project Introduction

The relentless advancement of web technologies and the increasing complexity of web applications have underscored the importance of robust cybersecurity measures, particularly in the realm of memory safety. Traditional programming languages like C and C++ have long been the backbone of system and application development. However, they are notoriously susceptible to memory vulnerabilities, which can lead to significant security breaches. This research paper aims to provide a comprehensive assessment of these traditional memory vulnerabilities in both native C/C++ environments and the emerging platform of WebAssembly—a binary instruction format for a stack-based virtual machine, aimed at enabling high-performance applications on the web.

Simultaneously to this, the exploration into the memory safety features of modern programming languages, Rust and Go, forms a cornerstone of our study. These languages have been designed with built-in mechanisms to mitigate common memory safety issues, promising a safer development environment. Through a thorough identification of vulnerabilities via Common Weakness Enumeration (CWE) analysis, this paper aims to bridge the gap between theoretical knowledge and practical application, providing coding examples in multiple languages to illustrate the nuances of memory safety across different programming environments.

A significant portion of our research is devoted to testing these vulnerabilities within the WebAssembly environment, with a particular focus on assessing the memory safety enhancements offered by Rust and Go. This investigation is not only pivotal for understanding

the security implications of deploying WebAssembly applications but also for evaluating the potential of these modern languages in elevating the security posture of web applications at large.

This paper also explores the integration of Artificial Intelligence (AI) tools, notably ChatGPT, in the process of vulnerability detection and remediation. The effectiveness of these AI-powered tools in parsing and analyzing source code for potential security vulnerabilities offers a promising avenue for enhancing the security of software development workflows. By automating the detection and mitigation of memory vulnerabilities, AI tools have the potential to significantly reduce the incidence of exploitable security flaws in software applications.

This research is driven by the goal of providing a holistic overview of the current landscape of memory vulnerabilities in software development, with a keen eye on the future of memory safety in web technologies. Through the provision of empirical evidence and practical insights, this paper aims to contribute valuable knowledge to the fields of cybersecurity and software development, empowering developers, security professionals, and researchers with the tools and understanding necessary to forge safer, more secure software for the future.

1.1 Top-Level Research Project Description

This project assesses the effectiveness of traditional C/C++ memory mitigations in WebAssembly and explores the memory safety features of Rust and Go in this context. By identifying vulnerabilities through CWE analysis, coding examples in multiple languages, and testing in WebAssembly, the project aims to determine if Rust and Go provide superior memory safety. Integration of AI tools like ChatGPT will further evaluate their effectiveness in detecting and remediating vulnerabilities in the source code.

1.2 Project Objectives / Goals

In the evolving landscape of web technologies, the security of WebAssembly applications has emerged as a pivotal concern. This project report details our comprehensive examination of the viability of conventional migration strategies to address vulnerabilities inherent in WebAssembly. Simultaneously, it highlights the security advantages provided by memory-safe programming languages such as Rust and Go. Through the inclusion of real-world case studies, we illustrate the potential for successful vulnerability mitigation and provide a well-rounded perspective on enhancing security in WebAssembly development.

Examination of Migration Strategies

Our research began with an in-depth examination of conventional migration strategies traditionally used to address vulnerabilities in software development. We evaluated these strategies within the unique context of WebAssembly, assessing their effectiveness in mitigating security risks. This evaluation aimed to understand how traditional approaches could be adapted or evolved to better suit the needs of WebAssembly security.

Emphasis on Rust and Go

A significant portion of our study was devoted to exploring the security benefits of Rust and Go. These modern programming languages are designed with memory safety as a core principle, offering built-in mechanisms to prevent common vulnerabilities. By analyzing the features and security models of Rust and Go, we shed light on how these languages can serve as powerful tools in enhancing the security posture of WebAssembly applications.

Successful Vulnerability Mitigation

Our findings demonstrate successful examples of vulnerability mitigation within the WebAssembly context. Through detailed analysis of specific vulnerabilities and the strategies employed to address them, we highlight the effectiveness of both conventional migration strategies and the adoption of Rust and Go. These success stories underscore the feasibility of enhancing security in WebAssembly through thoughtful application of proven and innovative methods.

Key Insights and Recommendations

The culmination of our research is presented in a set of key insights and recommendations aimed at developers, security professionals, and decision-makers involved in WebAssembly development. We advocate for the adoption of memory-safe languages, such as Rust and Go, in projects where security is of paramount importance. Furthermore, our recommendations include guidance on integrating conventional migration strategies with the unique aspects of WebAssembly to achieve optimal security outcomes.

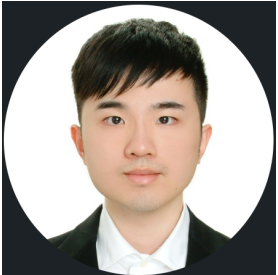
Advocacy for Memory-Safe Languages

Throughout this report, we strongly advocate for the adoption of memory-safe programming languages in security-sensitive projects. The evidence gathered through our research supports the conclusion that Rust and Go can play a critical role in minimizing vulnerabilities and enhancing the overall security of WebAssembly applications.

Conclusion

Our aim has been to provide a comprehensive and nuanced perspective on the challenges and solutions related to security in WebAssembly development. By examining conventional migration strategies, highlighting the benefits of Rust and Go, and presenting real-world case studies, we offer valuable insights and practical recommendations for improving security practices. As WebAssembly continues to grow in popularity, the adoption of memory-safe languages and effective migration strategies will be key to safeguarding the future of web application security.

1.3 Investigator Biographies

 Joshua McCoy	Education: -B.S. in Computer Science, Miami University, Oxford, Ohio. Graduate Student in Cybersecurity, University of Arizona. Experience: -Systems Security Engineer at RTX, Tucson, AZ, focusing on PIT System Cybersecurity and Anti-Tamper for missile systems. -Software Engineering intern at Microsoft (React.js). -Data Science intern at The Center for Analytics and Data Science (Data Analytics, R, RStudio). -Software Developer for The Miami Tribe of Oklahoma (HTML, CSS, JavaScript, SQL). -Associate Developer Consultant at Cyborg Mobile (React.js).
 Chia-Hao Yang	Education: -B.S. in Electrical Engineering. -M.S. in Computer Science (in progress) at the University of Texas at Dallas. -Graduate Certificate in Cyber Defense (NSA CAE). Experience: -Top 10% in NSA Codebreaker Challenge 2022. -3rd place in TexSAW 2022 CTF competition.

 Nate Schultz	Education: -B.A. in Information Communication Technology and Policy, University of Hawaii. -M.S. in Management Information Systems, University of Arizona. -Currently completing a Cybersecurity Master's at the University of Arizona. Experience: -15 years in IT, with the last 3 years as a System Administrator at RTX.
 Ryan Brockinton	Education: -B.S. in Computer Science from the University of Texas at Dallas. -Pursuing an M.S. in Computer Science with an emphasis in Cybersecurity at the same university. Experience: -Participated in Glacier CTF 2023, focusing on reverse engineering, low-level programming, and web exploitation. -Multiple internships at Texas Instruments, focusing on system automation and web development.

2.0 Research Plan and Progress to Date

In the course of our research into memory Common Weakness Enumerations (CWEs) relevant to WebAssembly, we have made significant strides in identifying and cataloging vulnerabilities that are particularly relevant to our study. Given the expansive landscape of vulnerabilities associated with C and C++ programming languages, our initial task was to discern which of these vulnerabilities are also applicable to WebAssembly environments. This process was crucial, as not all vulnerabilities in C and C++ translate directly to WebAssembly due to its unique execution model and security features.

Identification of Relevant CWEs

Through analysis, our team has refined the list of potential vulnerabilities down to five key CWEs that are applicable and significant to our research on WebAssembly. This focus

enables us to conduct a more targeted and in-depth investigation into these vulnerabilities, assessing their impact within WebAssembly contexts and exploring effective mitigation strategies.

The identified CWEs are as follows:

CWE-476: NULL Pointer Dereference - This vulnerability occurs when the application dereferences a pointer that it expects to be valid but is NULL, leading to a crash or unintended behavior.

CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer - This category of vulnerability encompasses errors that occur when operations are performed on a memory buffer, but the operations do not properly respect the buffer's boundaries, potentially leading to unauthorized access or information disclosure.

CWE-787: Out-of-bounds Write - This involves writing data past the boundaries of allocated memory, which can result in the corruption of data, crash of the system, or code execution.

CWE-121: Stack-based Buffer Overflow - A specific instance of buffer overflow occurring on the stack, this vulnerability can lead to arbitrary code execution, denial of service, or escalation of privileges.

CWE-122: Heap-based Buffer Overflow - Similar to the stack-based buffer overflow, but occurring on the heap, this vulnerability can also lead to serious security breaches such as arbitrary code execution.

Hierarchical Relationship Among Identified CWEs

Of particular interest is the hierarchical relationship observed among the identified CWEs. Two of the vulnerabilities, CWE-121 and CWE-122, are considered "child" vulnerabilities of the broader category represented by CWE-119. This distinction is critical for our analysis, as it highlights the nuanced differences in how buffer overflow vulnerabilities manifest in stack and heap memory and their relevance to WebAssembly's security model.

Conclusion

The progress to date in identifying and categorizing the CWEs applicable to WebAssembly represents a significant milestone in our research. By focusing on these five key vulnerabilities, we aim to dive deeper into the mechanics of these security issues within the WebAssembly environment, test their exploitability, and develop comprehensive mitigation strategies. This targeted approach not only streamlines our research efforts but also enhances the potential impact of our findings on the security of WebAssembly applications.

2.1 Research Plan Summary

The proposed research plan aims to delve into the realm of memory Common Weakness Enumerations (CWEs) specifically within the WebAssembly framework. This study is set to explore the intricacies of security vulnerabilities that emerge in the WebAssembly context, contrasting them with their manifestations in native environments. The research is structured to encompass both theoretical and practical facets of cybersecurity, ensuring a holistic approach to understanding and mitigating vulnerabilities.

Research Objective: The primary objective is to investigate the prevalence, nature, and mitigation of memory CWEs in WebAssembly, comparing and contrasting these findings with traditional native environments. This involves an extensive review of current literature, as well as hands-on experiments to expose and address vulnerabilities.

Division of Tasks: The research team will be split into two primary groups: the programming team and the research team. The programming team will focus on crafting C/C++ examples that demonstrate exploitable vulnerabilities in both WebAssembly and native environments. Concurrently, the research team will undertake a comprehensive study of WebAssembly-related

vulnerabilities as documented in the National Vulnerability Database (NVD), providing a theoretical foundation for the practical experiments.

Methodology: The research methodology encompasses several key phases:

- **Creation of Vulnerable C/C++ Examples:** The programming team will develop examples in C/C++, aimed at illustrating specific vulnerabilities in both WebAssembly and native code contexts.
- **Study of WebAssembly Cases in NVD:** Parallel to the programming efforts, the research team will analyze documented WebAssembly vulnerabilities, offering insights into historical and emerging security challenges.
- **Migration Strategies and Literature Review:** Traditional mitigation strategies employed in native environments will be reviewed and tested for their applicability to WebAssembly. This phase includes a literature review to understand the theoretical underpinnings of these strategies.
- **Experimental Testing:** The feasibility of traditional mitigation strategies within the WebAssembly environment will be empirically tested, allowing for direct observation of their effectiveness.
- **Code Recreation and AI Tool Evaluation:** C/C++ code will be recreated using Rust and Go to assess the efficacy of AI tools in detecting and remediating vulnerabilities. This phase aims to evaluate the potential of modern programming languages and AI in enhancing cybersecurity measures.
- **Statistical Analysis:** A comprehensive statistical analysis will be conducted on the results obtained from testing different AI tools, aiming to quantify their effectiveness in securing systems against identified vulnerabilities.

Expected Outcomes: The culmination of this research is expected to yield a nuanced understanding of memory CWEs in WebAssembly, comparative insights between WebAssembly and native environments, and a critical evaluation of traditional and innovative mitigation strategies. Moreover, it aims to contribute to the cybersecurity field by offering evidence-based

recommendations for the deployment of AI tools in vulnerability detection and remediation efforts.

Schedule

January 23, 2024	Project Bids
January 26, 2024	Project assignments
February 4, 2024	Project proposal
February 5 - February 12, 2024	Research memory CWE's that happen in WebAssembly
February 13 - February 26, 2024	Implement C/C++ examples exploiting in native environment and WebAssembly
February 26 - March 11, 2024	Implement traditional migration in native environment
March 11 - March 21, 2024	Testing traditional migration availability in WebAssembly
March 10- March 21, 2024	Midterm report writing and presentation practice run
March 22, 2024	Midterm presentations / Midterm deliverables

2.2 Deliverables

This segment of the report delineates the deliverables produced over the course of our research into memory Common Weakness Enumerations (CWEs) in WebAssembly. Our project was designed to not only advance the understanding of security vulnerabilities within this domain but also to provide tangible tools and resources to the academic and industry

communities for further exploration and application. Below is an overview of the key deliverables resulting from this project:

- **Comprehensive Final Report**

The cornerstone of our project deliverables is a comprehensive final report that encapsulates the entirety of our research journey. This document is meticulously structured to cover the research methodologies employed, detailed findings from our investigations, and the derived insights. It provides an exhaustive narrative of our exploration into memory CWEs in WebAssembly and their implications in both theoretical and practical contexts.

- **Professionally Crafted Presentation Materials**

To effectively disseminate our findings and insights to both academic and industry audiences, we have developed a suite of professionally crafted presentation materials. These materials are designed to convey the complexities and nuances of our research in an accessible and engaging manner, facilitating knowledge transfer and stimulating discussion among scholars, developers, and cybersecurity practitioners.

- **Dedicated Repository**

Understanding the importance of transparency and reproducibility in research, we have established a dedicated repository to house all project-related materials. This repository serves as a central hub for accessing our research methodologies, findings, presentation materials, and any supplementary documents or resources.

- **Curated Collection of Test Cases**

Within the dedicated repository, we have included a curated collection of test cases that were instrumental in our experimental investigations. These test cases are provided with

comprehensive documentation to facilitate replication of experiments and validation of findings by others in the field. This resource aims to support ongoing research and development efforts in understanding and mitigating memory CWEs in WebAssembly.

- **Analysis Tools**

Another significant contribution to the repository is the suite of analysis tools developed over the course of our project. These tools, which were integral to our research processes, are made available for public use. They enable scholars, developers, and cybersecurity practitioners to conduct their own analyses, potentially extending the scope of our initial research or applying the tools in new and innovative contexts.

Aim and Impact

The overarching aim of these deliverables is to empower a broad spectrum of stakeholders, from scholars and developers to cybersecurity practitioners, with robust resources that not only validate the outcomes of our project but also provide a foundation for building upon these results. Through the dissemination of our final report, presentation materials, and the provisioning of a dedicated repository with test cases and analysis tools, we aspire to make a lasting contribution to the field of cybersecurity, specifically in the domain of WebAssembly security vulnerabilities.

2.3 Progress to Date

This section of our report outlines the progress made in our investigation into memory Common Weakness Enumerations (CWEs) in WebAssembly up to the current point. Our team has embarked on a comprehensive journey to understand the landscape of security vulnerabilities, particularly those affecting C/C++ environments and how they translate to the

WebAssembly context. Here, we detail the steps taken and the groundwork laid in pursuit of our research objectives.

- **Review of Previous Projects**

Our initial step involved a thorough examination of previous projects undertaken by teams within our domain. This retrospective analysis was instrumental in understanding the scope of work previously explored, identifying gaps in knowledge, and ensuring that our research builds upon and extends the existing body of knowledge. This process not only helped in avoiding duplication of efforts but also in pinpointing areas ripe for innovation.

- **Investigation of C/C++ Vulnerabilities**

Given that C/C++ vulnerabilities form a significant portion of the security issues that potentially affect WebAssembly, we dedicated a substantial part of our efforts to scrutinizing these vulnerabilities. Our team dissected various known vulnerabilities within these languages to comprehend their nature, impact, and the mechanisms by which they are exploited. This examination is crucial for our later work in testing these vulnerabilities within the WebAssembly environment.

- **Researching Vulnerabilities in NVD and CWE Databases**

To deepen our understanding of the vulnerabilities relevant to our study, we conducted extensive research into the National Vulnerability Database (NVD) and the Common Weakness Enumeration (CWE) databases. This exploration allowed us to catalog and analyze vulnerabilities that are directly applicable to our focus on WebAssembly, facilitating a targeted approach to our investigation.

- **WebAssembly Environment Testing**

The testing compiler used in the native environment for C code is GNU Compiler Collection (GCC); The testing compiler used in WebAssembly environment is Emscripten Compiler Frontend (EMCC).

- **Memory Dynamic Analysis Tool: C/C++ DevTools Support (DWARF)**

A DevTools Plugin for debugging C/C++ WebAssembly applications (using DWARF debug information), provided by Chrome DevTools Team. It helps to trace memory usage in browsers like GDB in the native environment.

- **CWE-121: Stack-based Buffer Overflow:**

Compared to the native environment, the WebAssembly environment lacks detection of stack overflows. GCC compiled binaries by default abort execution when a stack overflow is detected, while in EMCC compiled binaries, a stack buffer declared later can overflow into a stack buffer declared earlier. This result shows that the WebAssembly environment has a corresponding risk of data corruption. As an additional research finding, we discovered that the WebAssembly environment clears the binary code before allocating memory to the stack section, whereas the native environment may leave behind uncleared binary code.

<pre> argv[1]: a_20bytes_1234567890 Execute: char buf1[16], buf2[16]; strcpy(buf1, "password"); Stack Data 1: password, Memory Address: 0x16b1b3388 Stack Data 2: 04, Memory Address: 0x16b1b3378 Execute: strcpy(buf2, argv[1]); Trace/BPT trap: 5 </pre>	<pre> argv[1]: a_20bytes_1234567890 Execute: char buf1[16], buf2[16]; strcpy(buf1, "password"); Stack Data 1: password, Memory Address: 0xffa0 Stack Data 2: , Memory Address: 0xff90 Execute: strcpy(buf2, argv[1]); Stack Data 1: 7890, Memory Address: 0xffa0 Stack Data 2: a_20bytes_1234567890, Memory Address: 0xff90 </pre>
Native Environment	WebAssembly Environment

- **CWE-122: Heap-based Buffer Overflow:**

Compared to the native environment, the WebAssembly environment lacks corresponding mitigations against heap overflows due to memory allocation deficiencies, resulting in a relatively much higher risk of being attacked by corresponding methods. From the testing cases, it is known that in the default mode, GCC compiled binaries provide memory address space layout randomization mitigation and a 416-byte gap between different heap buffers, greatly reducing the risk of heap buffer overflow. However, in the WebAssembly environment, the memory location of each heap buffer is fixed during execution, with only a 16-byte gap. As a result, simple heap overflow attacks that have been prevented in the native environment can still be effective in the WebAssembly environment.

<pre> argv[1]: a_20bytes_1234567890 argv[2]: b_20bytes_1234567890 Execute: char *ptr1, *ptr2; ptr1 = malloc(8); ptr2 = malloc(8); Heap Data 1: , Memory Address: 0x151605e60 Heap Data 2: , Memory Address: 0x151606000 Execute: strcpy(ptr1, argv[1]); Heap Data 1: a_20bytes_1234567890, Memory Address: 0x151605e60 Heap Data 2: , Memory Address: 0x151606000 Execute: strcpy(ptr2, argv[2]); Heap Data 1: a_20bytes_1234567890, Memory Address: 0x151605e60 Heap Data 2: b_20bytes_1234567890, Memory Address: 0x151606000 </pre> Native Environment	<pre> argv[1]: a_20bytes_1234567890 argv[2]: b_20bytes_1234567890 Execute: char *ptr1, *ptr2; ptr1 = malloc(8); ptr2 = malloc(8); Heap Data 1: , Memory Address: 0x10a88 Heap Data 2: , Memory Address: 0x10a98 Execute: strcpy(ptr1, argv[1]); Heap Data 1: a_20bytes_1234567890, Memory Address: 0x10a88 Heap Data 2: 7890, Memory Address: 0x10a98 Execute: strcpy(ptr2, argv[2]); Heap Data 1: a_20bytes_123456b_20bytes_1234567890, Memory Address: 0x10a88 Heap Data 2: b_20bytes_1234567890, Memory Address: 0x10a98 </pre> WebAssembly Environment
--	---

- **CWE-476: NULL Pointer Dereference:**

In both the native and WebAssembly environments null pointer dereferences work similarly within C. At compile time the GCC or emcc (WebAssembly compiler) doesn't present any warnings to warn of the null pointer which leads to the program crashing at runtime. Both compilers have flags, options to customize the compile process of the file,

which could be used to try to detect the fault but there are test cases where these flags do not detect the fault. From the testing cases, the native environment produces a segmentation fault when the null pointer is dereferenced due to the program trying to access memory location zero which is a restricted address whereas WebAssembly aborts the program due to the same cause. On the other hand C++'s object oriented design allows for more undefined behaviors. When a null pointer dereference occurs on a class object a segmentation fault could occur if a function is called within the object that references a virtual function or a member variable which would require the pointer to be dereferenced but if neither a virtual function or member variable is referenced within that called function the optimizations of the compiler bypass the dereference and could appear to be successful leading to undefined behavior.

- **Conducting Literature Reviews**

Concurrent with our practical investigations, we have engaged in comprehensive literature reviews to underpin our research with a solid theoretical foundation. These reviews span a wide range of topics, from the specifics of WebAssembly security to broader concepts in cybersecurity and programming languages. The insights gained from these reviews have been invaluable in shaping our research direction and methodologies.

Conclusion

To date, our progress lays a robust foundation for the next phases of our project. Through a blend of practical investigations and theoretical inquiries, we have positioned ourselves to dive deeper into the nuances of WebAssembly vulnerabilities and their mitigations. Our team's

dedication to expanding our knowledge base and skill set, particularly in new programming languages, sets the stage for innovative approaches to securing WebAssembly applications.

3.0 Summary of Results/Conclusions to Date

In our literature research and review we have found a few things. Extensive literature exists on C/C++ memory vulnerabilities and surprisingly there is also a good amount of literature on WebAssembly memory vulnerabilities. Literature reviews and research have shown our desire to test WebAssembly alongside C/C++ compared to “Memory Safe” languages is a quite worthy project. WebAssembly is implemented in 92% of world wide browsers and thus is being used by many users who don’t even realize it. Some CVE’s we found are not applicable in WebAssembly (even if they are in C or C++) so this led us to tailoring our vulnerability selection to those we could test in both WebAssembly and C languages.

- **CWE-121: Stack-based Buffer Overflow:**

WebAssembly lacks stack overflow detection compared to native environments, leading to potential data corruption risks. EMCC compiled binaries may allow stack buffers declared later to overflow into earlier ones. Additionally, in WebAssembly, binary code is cleared before allocating memory to the stack section, unlike in native environments where uncleared binary code may remain.

- **CWE-122: Heap-based Buffer Overflow:**

Compared to native environments, WebAssembly lacks effective mitigations against heap overflows, making it more vulnerable to corresponding attacks. GCC compiled binaries offer protections like memory address space layout randomization and a 416-byte gap between heap buffers, significantly reducing the risk of overflow. However, in WebAssembly, heap buffer

locations are fixed during execution with only a 16-byte gap, allowing simple overflow attacks to remain effective.

- **CWE-476: NULL Pointer Dereference:**

Compared to native environments, WebAssembly contains the same flaws against null pointer dereferences. Both environments have difficulties detecting the faults at compile time which can lead to crashes or undefined behaviors of the program at runtime. Although the risk of program crashes is significant and apparent, the risk of the visible success or undefined behaviors of the program can remain unknown. For this fault WebAssembly does not provide any more protections or mitigations than the native environment creating the same risk for the faults to occur.

4.0 Issues and Challenges

Luckily our issues and challenges have been kept to a minimum but there are a few to list. Issues have included problems adapting software vulnerabilities into new languages, which has been seen in that some CWEs do not have the ability to be converted into WebAssembly, and the response time for our remote virtual machine has been slower than expected. We have been mitigating these issues via early testing as well as bringing in subject matter experts as needed to help. There have been more challenges for our project but these can be good for the project's development. Challenges have included: having to work through an iterative refinement of the software test cases, working to ensure the accuracy of the vulnerability comparisons, team members inexperienced with the new software languages, and the fact that some team members lack a computer science/software development background. We have been mitigating our challenges through the proper delegation of tasks to match group members backgrounds and

utilizing any needed external sources to make sure we are utilizing and developing our software correctly.

5.0 Bibliography

Daniel Lehmann, Johannes Kinder, Michael Pradel (2020). Everything Old is New Again: Binary Security of WebAssembly. *the Proceedings of the 29th USENIX Security Symposium*, pg 217 - p 234 . Publisher. <https://www.usenix.org/system/files/sec20-lehmann.pdf>

C. Disselkoen, J. Renner, C. Watt, T. Garfinkel, A. Levy, and D. Stefan (2019). Position paper: Progressive memory safety for webassembly. *Proceedings of the 8th International Workshop on Hardware and Architectural Support for Security and Privacy*, pg 1 - pg 8. Publisher. <https://doi.org/10.1145/3337167.3337171>

Quentin Stiévenart, Coen De Roover (2021). Mohammad GhafariThe Security Risk of Lacking Compiler Protection in WebAssembly. *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. Publisher. <https://doi.org/10.1109/QRS54544.2021.00024>

Suhyeon Song, Seonghwan Park, Donghyun Kwon (2023). metaSafer: A Technique to Detect Heap Metadata Corruption in WebAssembly. *IEEE Access*, volume: 11, pg 124887 - pg 124898. Publisher. <https://doi.org/10.1109/ACCESS.2023.3327817>

Minseo Kim, Hyerean Jang, Young-joo Shin (2022). Avengers, Assemble! Survey of WebAssembly Security Solutions. *2022 IEEE 15th International Conference on Cloud Computing*. Publisher. <https://doi.org/10.1109/CLOUD55607.2022.00077>

Marius Musch, Christian Wressnegger, Martin Johns, Konrad Rieck (). New Kid on the Web: A Study on the Prevalence of WebAssembly in the Wild. *Detection of Intrusions and Malware, and Vulnerability Assessment. DIMVA 2019. Lecture Notes in Computer Science()*, vol 11543. Springer, Cham. Publisher. https://doi.org/10.1007/978-3-030-22038-9_2

Appendix A: Acronyms and Terms/ Definitions

Acronym	Definition
---------	------------

AI	Artificial Intelligence
CWE	Common Weakness Enumeration
CTF	Capture The Flag (inferred context, not explicitly defined)
EMCC	Emscripten Compiler Frontend (inferred context, not explicitly defined)
GDB	GNU Debugger
GCC	GNU Compiler Collection (inferred context, not explicitly defined)
NSA	National Security Agency
NVD	National Vulnerability Database
PIT	Platform IT
Wasm	WebAssembly

- **AI (Artificial Intelligence):** Refers to the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions. AI can be applied to various sectors including software development for tasks such as vulnerability detection and remediation.
- **CWE (Common Weakness Enumeration):** A list that categorizes common software weaknesses and vulnerabilities. It serves as a standard for identifying, mitigating, and preventing software vulnerabilities.
- **CTF (Capture The Flag):** In cybersecurity, it refers to a competition where participants solve security-related challenges to find a specific piece of data, called the "flag." This term was inferred and not explicitly defined in the provided text.
- **EMCC (Emscripten Compiler Frontend):** A tool used to compile C and C++ code into WebAssembly. It allows developers to run C and C++ code on the web at near-native

speed.

- **GDB (GNU Debugger):** A debugger that provides extensive facilities for tracing and altering the execution of computer programs. It's part of the GNU Project and supports various programming languages including C and C++.
- **GCC (GNU Compiler Collection):** A compiler system produced by the GNU Project supporting various programming languages. It's standard for compiling C and C++ code, among others.
- **NSA (National Security Agency):** A U.S. government agency responsible for national security and the protection of government communications and information systems against penetration and network warfare.
- **NVD (National Vulnerability Database):** A database that contains information about security-related software flaws, misconfigurations, product names, and impact metrics. It's managed by the National Institute of Standards and Technology (NIST).
- **PIT (Platform IT):** Refers to information technology solutions and systems that support specific platforms or operations, often in a specialized or embedded context. This term was inferred and not explicitly defined in the provided text.
- **Wasm/WebAssembly:** A binary instruction format for a stack-based virtual machine. WebAssembly is designed as a portable compilation target for high-level languages like C/C++ and Rust, enabling deployment on the web for client and server applications.